AL-AQSA UNIVERSITY

Faculty of Information Technology

Department of Computer Science

HUSSEIN AHMED GRADUATION PROJECT

Design and Implementation of an Arabic School Management System

مشروع تخرج حسين أحمد



Hussein Tariq Hussein Ahmed

Student ID 1320222455

Supervised by

Dr. Abdul Basit Abdullah Al-Masri

A project report submitted in partial fulfilment of the requirements for the award of

Bachelor of Computer Science

2026/2027

28 October 2026

COPYRIGHT

Copyright © 2026 Hussein Tariq Hussein Ahmed. Rights in the original report and project material belong to their respective authors. External documentation and scholarly material are acknowledged through citations and references. The project package contains fictional demonstration records.

DECLARATION

I declare that I have reviewed this project report and take responsibility for the submitted work, including the acknowledgement of external sources and any assistance that must be disclosed under university policy. This report has not been submitted concurrently for another award. I will sign this declaration only after verifying that it accurately describes my submission.

Field	Student declaration
Name	Hussein Tariq Hussein Ahmed
Student ID	1320222455
Date of report	28 October 2026
Signature	To be signed personally by the student at submission

APPROVAL FOR SUBMISSION

Project title: Hussein Ahmed Graduation Project. Student: Hussein Tariq Hussein Ahmed. Degree: Bachelor of Computer Science, Al-Aqsa University. This page is reserved for the supervisor’s certification that the report meets the required standard for submission. Inclusion of this page does not itself record approval.

Field	Supervisor certification
Supervisor	Dr. Abdul Basit Abdullah Al-Masri
Signature	To be signed by the supervisor after review
Approval date	To be dated by the supervisor after review

ACKNOWLEDGEMENTS

I thank Allah for the ability to pursue this work. I thank my family for their support and the faculty of the Department of Computer Science at Al-Aqsa University for the knowledge developed during my studies. I acknowledge my supervisor, Dr. Abdul Basit Abdullah Al-Masri, and the authors of the documentation and research cited in this report.

ABSTRACT

This project presents an Arabic-first school management prototype that links student records, academic placement, attendance, marks and financial workflows. The application uses a browser interface, Python request handling and SQLite storage, with local HTTP operation and a PythonAnywhere WSGI alternative. Its core controls preserve archived student history, support additional memberships, keep receipts in draft until explicit print confirmation, separate cash debt payments from monthly fees and record only manually sent communication lists. The evaluation inspected the implementation and executed four isolated integration scenarios. All four passed, and the supplied demonstration database returned a successful integrity check without foreign-key violations. These results establish selected functional behaviour, not measured organizational benefit or production readiness. The report provides requirements, UML models, a selective ERD and a traceability matrix. Remaining work includes authenticated-screen documentation, staff usability evaluation, wider access-control testing, precise monetary representation and independent security review. The demonstration data are fictional.

Keywords: school information system, Python, SQLite, WSGI, Arabic interface, student finance, integration testing.

الملخص

يقدم المشروع نموذجًا أوليًا عربيًا لإدارة المدرسة يربط بيانات الطلاب والتسجيل والحضور والعلامات والعمليات المالية. يستخدم واجهة متصفح وخادم بايثون وقاعدة إس كيو لايت، ويدعم التشغيل المحلي أو الاستضافة عبر بايثون أني وير. يحفظ تاريخ الطالب المؤرشف، ويدعم العضويات الإضافية، ويفصل الديون عن الرسوم، ويشترط تأكيد الطباعة لاعتماد الإيصال. تقتصر الرسائل على إعداد قوائم للإرسال اليدوي. نجحت أربعة سيناريوهات لاختبار التكامل وفحص سلامة قاعدة العرض، وهي نتائج تثبت سلوكًا وظيفيًا محددًا ولا تثبت الجاهزية المؤسسية الكاملة. يتضمن التقرير المتطلبات والمخططات والتصميم والتنفيذ وحدود التقييم. تحتاج المرحلة التالية إلى استكمال صور الواجهات بعد الدخول، وتقييم الاستخدام مع الموظفين، وتوسيع اختبارات الصلاحيات، وتحسين تمثيل القيم المالية. جميع بيانات العرض وهمية.

TABLE OF CONTENTS

CHAPTER ONE INTRODUCTION 12
1.1 Introduction 12
1.2 Problem Statement 12
1.3 Project Aim and Objectives 13
1.4 Project Scope 13
1.5 Significance of the Project 14
1.6 Proposed Solution and Project Overview 14
1.7 Methodology Overview 15
1.8 Report Organization 15
1.9 Limitations and Constraints 15
CHAPTER TWO LITERATURE REVIEW 16
2.1 Introduction 16
2.2 Theoretical Background 16
2.4 Related Systems and Existing Solutions 17
2.5 Previous Studies 18
2.6 Comparative Analysis 19
2.7 Research Gap and Identified Limitations 20
2.8 Chapter Summary 20
CHAPTER THREE METHODOLOGY 21
3.1 Introduction 21
3.2 Methodology 21
3.3 Related Technologies and Tools 22
3.4 Chapter Summary 23
CHAPTER FOUR DESIGN AND IMPLEMENTATION 24
4.1 Introduction 24
4.2 Requirements Analysis 24
4.3 Data Collection and Requirements Elicitation 26
4.4 System Requirements and Feasibility 26
4.5 System Modeling and Design 27
4.6 Database Design 30
4.7 User Interface Design 32
4.8 System Implementation 33
4.9 Database Implementation 33
4.10 User Interface Implementation 34
4.11 System Testing 35
4.12 Chapter Summary 36
CHAPTER FIVE CONCLUSION AND RECOMMENDATIONS 37
5.1 Introduction 37
5.2 Summary of Key Findings 37
5.3 Contributions to the Field 37
5.4 Recommendations 38
5.5 Recommendations for Future Research 38
REFERENCES 39
APPENDICES 40

LIST OF FIGURES

Figure 4.1 Use case diagram

Figure 4.2 Conceptual domain class diagram

Figure 4.3 Payment and receipt confirmation sequence

Figure 4.4 Receipt state diagram

Figure 4.5 Manual communication activity diagram

Figure 4.6 Selective physical entity relationship diagram

Figure 4.7 Interface layout wireframe

Figure 4.8 Actual hosted Arabic login screen

LIST OF TABLES

Chapter One: objective acceptance criteria. Chapter Two: existing-system comparison and study comparison. Chapter Three: development stages and technology choices. Chapter Four: functional requirements, non-functional requirements, core entities, test results and requirements traceability. Appendix B: full database table inventory.

LIST OF ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
ERD	Entity Relationship Diagram
HTTP / HTTPS	Hypertext Transfer Protocol / secure transport
SIS	Student Information System
UML	Unified Modeling Language
WSGI	Web Server Gateway Interface
PK / FK	Primary Key / Foreign Key
HSTS	HTTP Strict Transport Security
NIS	Israeli new shekel, the currency label used by the delivered interface

CHAPTER ONE INTRODUCTION

1.1 Introduction

School administration depends on consistent information about students, families, academic placement, attendance and payments. A student may appear in a registration book, a spreadsheet of fees and a separate teacher attendance sheet. When these records use different identifiers or are updated at different times, staff must reconcile them manually. A student information system connects these activities through a common student identity and explicit rules for updating records.

This project addresses the administration of a small school through an Arabic-first web interface and a Python application backed by SQLite. The project title is Hussein Ahmed Graduation Project, and its practical output is a school management application. The system can run on a local computer without an Internet connection or through a PythonAnywhere WSGI deployment. These are alternative deployment modes of the same application. They do not provide automatic synchronization between two simultaneously active databases.

Within information technology, the project demonstrates how user-facing forms, request validation, relational data and deployment configuration work together. Its emphasis is the correctness of school workflows: retaining an archived student's history, distinguishing monthly fees from cash debts and requiring an explicit print-confirmation action before approving an issued receipt.

1.2 Problem Statement

The problem is the fragmentation of operational records and the ambiguity of actions applied to them. A payment recorded in one file may not be reflected in a receipt register. Removing a transferred student from an active list may also remove information needed to explain an earlier payment. A student attending an additional group may be duplicated to fit a single-class spreadsheet. A contact list prepared without review may include the wrong students or phone numbers.

The affected roles are administrators responsible for registration and accounts and teachers responsible for assigned classes, attendance and marks. Parents and students benefit indirectly from more consistent records, although the current system does not give them independent online portals. The proposed solution must connect records while keeping sensitive actions deliberate and visible.

This problem statement is grounded in the supplied application, its schema and its documented business rules. The evaluation does not establish the prevalence of these problems across Palestinian schools, nor does it quantify the time saved compared with a measured paper-based baseline. Such claims would require field data beyond the present technical evaluation.

1.3 Project Aim and Objectives

The aim is to deliver and evaluate an Arabic-first school management prototype that preserves student history and implements controlled academic and financial workflows in local and hosted operation.

Objective	Observable completion criterion
O1 Unify student records	Student and related records use StudentID and can be retrieved without creating duplicate student identities for extra classes.
O2 Preserve history	Archiving requires a reason, retains stored records and excludes the student from active financial views.
O3 Control receipts	A newly recorded payment produces a draft receipt; approval follows an explicit print-confirmation request.
O4 Separate cash debt	A debt has a principal and individual payments, with the remaining balance calculated from those payments.
O5 Review communication	The user filters and confirms recipients; the application records a manual-ready campaign without contacting an SMS provider.
O6 Support deployment	The local HTTP entry point and the WSGI adapter can access the same application functions under their respective configurations.

1.4 Project Scope

The implemented scope includes student and parent records, primary enrolment, additional class memberships, archive and restore operations, attendance, marks, monthly fees, additional fees, receipts, cash debts, manual contact campaigns, users, backups and reports. Administrator and teacher roles provide different access paths. Teacher permissions depend on assigned classes and subjects, rather than unrestricted access to administrative functions.

The scope excludes a parent self-service portal, online payment processing, automatic SMS dispatch, native mobile applications, machine-learning predictions, nationwide integration and bidirectional offline/cloud synchronization. The hosted configuration is a limited academic pilot. Large-scale concurrent operation, formal penetration testing and a statistically supported usability study are outside the completed evaluation.

1.5 Significance of the Project

The practical value is a coherent record of school operations that can be inspected and demonstrated. Separating a recorded payment from a printed receipt makes the meaning of each state explicit. Preserving archive metadata helps explain why a student is absent from current financial lists while historical transactions remain available. Additional memberships represent a student in more than one learning group without duplicating personal data.

The technical contribution is the integration of these rules into a small, reproducible Python application with an isolated test suite. This is an applied software contribution rather than evidence of a new database theory or a universally superior school platform. Its educational value lies in tracing a requirement through a model, an implementation path and an observable test.

1.6 Proposed Solution and Project Overview

The browser presents Arabic navigation, data-entry forms and filtered lists. JavaScript sends JSON requests to Python endpoints. The server authenticates protected requests, checks business constraints and reads or writes SQLite records. A Windows launcher supports local operation. The supplied WSGI adapter translates hosted web requests into the request format handled by the existing application, while PythonAnywhere provides the public web entry point.

Users first sign in, select a module and review its current records. Financial operations retain the student link throughout payment, receipt and debt handling. Communication stops at a confirmed recipient list. The user sends messages through a separate manual process, so the application does not represent a stored list as evidence that a message was delivered.

1.7 Methodology Overview

The development approach is iterative prototyping with requirement-to-test traceability. The existing workflows and schema were inspected, acceptance criteria were written for critical actions, the relevant paths were exercised and documentation was revised to reflect observed behaviour. The report uses the problem, design, demonstration and evaluation structure of design science as an organizing perspective (Peffers et al., 2007), without claiming that a controlled organizational research experiment was conducted.

1.8 Report Organization

Chapter Two reviews school information-system research, relevant technical concepts and existing products. Chapter Three explains the development method and technology choices. Chapter Four presents requirements, evidence sources, feasibility, UML models, database design, interface design, implementation and testing. Chapter Five summarizes findings, contributions, recommendations and research opportunities. References use APA author-date form. Appendices contain execution evidence, schema details and delivery instructions.

1.9 Limitations and Constraints

The evaluation uses fictional demonstration data and four automated integration scenarios. There are no participant interviews, questionnaire responses or measured usability results in the evidence set. The database and local server suit a small pilot; concurrent institutional use needs separate capacity and security evaluation. Hosted access depends on the Internet and hosting availability, while local access depends on the availability and security of the local computer. Local and cloud records can diverge if both are changed independently.

Resource limits include the graduation-project schedule, the absence of a dedicated operations team and the need to use a lightweight deployment. No benchmark-derived minimum hardware configuration or audited financial cost model is claimed. The feasibility estimates in Chapter Four are engineering planning assumptions rather than measurements of production demand.

CHAPTER TWO LITERATURE REVIEW

2.1 Introduction

This chapter connects the project with research on school information systems and with documentation for the technologies used. It distinguishes scholarly findings from product descriptions. The review considers administrative integration, access control, local deployment and the need to evaluate how a system is actually used. It then identifies a context-specific design need rather than claiming that existing platforms cannot perform school management.

The review is a focused narrative review, not a systematic review or meta-analysis. Two school-information-system studies and one methodology paper are discussed using accessible publisher or institutional records. Where only an abstract or bibliographic record was accessible, the discussion is limited to that evidence. Official product pages provide the basis for the existing-system comparison.

2.2 Theoretical Background

2.2.1 School information systems and operational integration

A school information system organizes records and supports administrative processes. The importance of adoption and actual use is separate from the mere presence of software. Visscher et al. (2003) examined implementation, use and effects of a computerized management information system in English secondary schools. Their research context motivates evaluation beyond a checklist of modules, although it cannot establish outcomes for the present school or application.

2.2.2 Browser applications and request boundaries

A browser interface separates presentation from data storage. HTML defines structure, CSS controls visual layout and JavaScript coordinates interaction. Python endpoints receive requests and apply authorization and validation before database operations. Interface validation can improve feedback, but the server must remain responsible for enforcing rules because a request can reach an endpoint independently of the displayed form.

2.2.3 Relational data and integrity

Relational design represents entities in tables and connects them using keys. A StudentID links enrolments, attendance and payments to the same student. Unique constraints prevent selected duplicates, while foreign keys constrain references. Python's sqlite3 module provides a DB-API interface and parameter binding for SQLite access (Python Software Foundation, n.d.). Application rules remain necessary when a relationship is conditional or spans several tables.

SQLite provides embedded storage without a separate database service. Its single-file model supports portability and local deployment. Its official guidance identifies local storage and many modest web workloads as appropriate uses, while advising client-server alternatives for workloads with different concurrency needs (SQLite, n.d.). This project therefore treats SQLite as a pilot choice, not a measured capacity guarantee.

2.2.4 Local operation and hosted operation

Local operation keeps the application and database on the user's computer and can continue without Internet access. Hosted operation exposes the application through an HTTPS website and a WSGI entry point. PythonAnywhere documents web-app configuration through its Web tab and WSGI setup (PythonAnywhere, n.d.). The two modes share application logic but have separate operational responsibilities, including database-path configuration and backups.

2.2.5 Workflow states and authorization

A state model defines which transitions change an object's meaning. For this project, draft and printed receipts are distinct states. A successful print-confirmation endpoint records the application's state change; it does not prove that paper physically emerged from a printer. Role-based access limits operations according to user role and assignments. Expiring sessions constrain the lifetime of an authenticated token, but do not replace secure hosting, device protection or independent security review.

2.4 Related Systems and Existing Solutions

Gibbon describes itself as a free, open-source school platform with shared access for staff, students and parents. Its documentation describes configurable access, student information and multilingual support, including right-to-left text. This makes it a

relevant reference for integrated school operation and Arabic interface needs. Its availability as open source does not eliminate deployment, maintenance and configuration effort (Gibbon, n.d.).

openSIS presents a broad student information platform covering student records, scheduling, attendance, grading, billing and communication. Its official feature catalogue describes role-based access and reporting, illustrating the breadth available in an established product. These advertised capabilities are a documentary comparison, not a hands-on security or performance audit of openSIS (openSIS, n.d.).

The present prototype offers a narrower combination tailored to its demonstration: Arabic screens, local Python operation, a hosted WSGI alternative and specific archive, receipt and manual-communication rules. The comparison does not establish that the reference systems lack equivalent configurable rules. Such a claim would require configuration-level testing.

2.5 Previous Studies

Visscher et al. (2003) reported a large-scale evaluation of a computerized management information system in English secondary schools. The study focuses on implementation, use and effects, making it relevant to the distinction between software availability and management benefit. The accessible record establishes the evaluation topic and setting. This report does not infer an unverified participant count, intervention design or numerical benefit. Its geographical setting and publication period limit direct transfer to an Arabic-first contemporary prototype.

Telem and Avidov (1995) examined changes associated with a school management information system in a high-school instruction-administration subsystem. Their preliminary study reports tighter hierarchical relationships in categories including accountability, evaluation, supervision, feedback, meetings and shared decision-making, while most horizontal relationships remained unchanged. The available abstract supports these reported directions, but not a claim of causation for this project. Its organizational focus and limited setting differ from the present technical validation.

Peffers et al. (2007) proposed a design science research methodology with problem identification, solution objectives, design and development, demonstration, evaluation and communication. They demonstrated the framework through four case studies. The paper helps structure an artifact-oriented report, but it does not validate this school's

requirements or replace application-specific tests. Here it informs the reporting logic while iterative prototyping describes the practical development work.

2.6 Comparative Analysis

Table 2.1 compares documented capabilities and evaluation scope. An unspecified capability means that this review did not verify it, not that the product lacks it.

Criterion	Gibbon official description	openSIS official description	Present prototype
Academic records	Integrated student and teaching platform	Student records, attendance and grading	Students, memberships, attendance and marks
User access	Configurable access for multiple stakeholder groups	Role-based permissions and portals	Administrator and assigned teacher roles
Arabic context	Right-to-left and multilingual support	Multilingual options advertised	Arabic-first interface
Local deployment used here	Not configured or tested in this review	Not configured or tested in this review	Python local HTTP operation demonstrated
Financial rules	No equivalent-rule test conducted	Billing and fees advertised	Separate cash debts and explicit receipt print gate
Evidence boundary	Vendor and documentation review	Vendor feature review	Code inspection and four integration tests

Table 2.2 compares the purpose of the selected studies with the present evaluation.

Study	Focus and evidence available	Relation to the project	Limit on transfer
Visscher et al. 2003	Implementation, use and effects in English secondary schools	Supports evaluation beyond module availability	Different period and institutional setting
Telem and Avidov 1995	Preliminary organizational study, abstract available	Shows that administrative information can relate to organizational practices	No local replication or causal measurement here

Study	Focus and evidence available	Relation to the project	Limit on transfer
Peffers et al. 2007	Methodology demonstrated through four cases	Organizes artifact design and evaluation	A methodology is not application evidence

2.7 Research Gap and Identified Limitations

The identified need is a small, reproducible Arabic-first implementation whose key business rules can be traced to tests and demonstrated both locally and through WSGI. It combines archive preservation, additional memberships, receipt approval after print confirmation, separate debt payments and manual-only communication in one inspectable package.

This is a contextual engineering gap. The reviewed products are broader systems, and the review does not establish worldwide novelty or product inferiority. The unanswered empirical question is whether these specific controls improve staff accuracy and task completion under local operating conditions. The current project establishes technical feasibility and selected functional correctness; a future field study is needed to establish user and organizational outcomes.

2.8 Chapter Summary

The literature distinguishes technical functionality from effective organizational use. Official documentation supports the selected web, SQLite and hosted deployment choices. Existing platforms demonstrate the breadth of school information systems, while the current prototype focuses on a bounded set of traceable rules. Chapter Three explains how these requirements were organized and evaluated through iterative development.

CHAPTER THREE METHODOLOGY

3.1 Introduction

This chapter explains the development method and the tools used to turn the problem statement into a working prototype. It identifies the evidence used during this revision and the limits of that evidence. The central methodological unit is a requirement linked to an implementation path and a repeatable acceptance check.

3.2 Methodology

Iterative prototyping was selected because financial and archival actions need observable behaviour before their wording can be documented accurately. A strictly sequential account would be misleading for an existing prototype that was inspected, exercised and revised. The method does not claim formal Scrum events, sprint velocities or a recorded Waterfall schedule that are not present in the project evidence. The first stage was problem and artifact inspection. The supplied code, database schema, demonstration generator, deployment notes and existing test suite were examined. The second stage converted rules into acceptance criteria, especially mandatory archive reasons, active-finance exclusion and the receipt print gate. The third stage examined the design through domain, interaction and state models. The fourth stage exercised the implementation against isolated data. The fifth stage reconciled report, presentation and delivery materials with the observed behaviour.
Table 3.1 records the practical outputs of these stages.

Stage	Input	Output or evidence
Inspect	Existing code, schema and delivery files	Module inventory and explicit scope
Specify	Critical business rules	Objectives O1-O6 and requirement identifiers
Model	Endpoints and relational entities	UML models and selective physical ERD
Exercise	Temporary database and local API	Four automated integration scenarios

Stage	Input	Output or evidence
Reconcile	Observed results and instructor outline	Five-chapter report, Arabic presentation and delivery package

The design science perspective helps explain the relationship between a practical problem and a constructed artifact (Peffers et al., 2007). However, the evaluation remains a technical case evaluation. No population sampling, statistical hypothesis test, interview study or randomized comparison is represented as completed.

3.3 Related Technologies and Tools

Python provides the HTTP handler, JSON processing, session checks, business rules and test execution. The sqlite3 library accesses relational data using the included schema. HTML, CSS and JavaScript provide the browser interface, and openpyxl supports optional Excel-related functions. WSGI is the interface used for PythonAnywhere hosting. Word and PDF provide the report outputs, and PowerPoint provides the Arabic discussion summary.

Decision	Selected option and reason	Alternative and trade-off
Application language	Python reuses the existing code and standard libraries	A different server framework would require migration and retesting
Database	SQLite supports a portable local pilot	PostgreSQL suits a separate server and higher concurrent-write demand
Interface	Browser-based Arabic interface needs no separate desktop client	A native mobile client would add platform-specific implementation
Deployment	Local HTTP plus hosted WSGI	A cloud-only system would not meet the offline requirement
References	APA author-date bibliography with RIS import data	Mendeley Cite insertion requires the user's authenticated desktop or Word session

Technology selection reflects the current scope rather than a benchmark proving one tool universally better. A future migration should begin with workload measurements

and preserve the existing acceptance criteria. Version-specific hosting settings must be checked against the actual PythonAnywhere account before applying deployment changes.

3.4 Chapter Summary

The methodology uses iterative inspection, specification, modeling and repeatable technical evaluation. It preserves a clear boundary between demonstrated application behaviour and unmeasured organizational benefits. Chapter Four now presents the requirements and models in detail and explains how the implementation satisfies selected acceptance criteria.

CHAPTER FOUR DESIGN AND IMPLEMENTATION

4.1 Introduction

This chapter connects requirements to design and implementation. It presents functional and non-functional requirements, explains how they were elicited, assesses feasibility and supplies the required UML and database models. It then describes the interface, source structure, data implementation and tests actually executed.

4.2 Requirements Analysis

4.2.1 Functional requirements

The system's functions are organized around a persistent student identity. The following matrix defines the acceptance behaviour of the central modules. It provides a stable basis for demonstration and later regression checks.

ID	Requirement	Acceptance condition
FR01	Manage student records	Create and retrieve a student with a unique student number.
FR02	Archive and restore	Reject an empty archive reason and preserve the student's stored history.
FR03	Support extra memberships	Return the same student in an additional assigned group without duplicating StudentID.
FR04	Manage monthly fees	Retrieve fee records with due and paid amounts and exclude archived students from active views.
FR05	Control receipt approval	Keep the receipt as draft until the print-confirm action sets IsPrinted.
FR06	Manage cash debts	Record a positive principal and positive payments against the correct debt.
FR07	Prepare manual campaigns	Require confirmation and store recipients without sending SMS.

ID	Requirement	Acceptance condition
FR08	Record learning data	Provide attendance and marks functions associated with the relevant student.
FR09	Authenticate users	Issue a session and reject expired authentication on protected routes.
FR10	Support operations	Provide database backup, reports and a health endpoint.

4.2.2 Non-functional requirements

ID	Requirement	Verification or present boundary
NFR01	Arabic usability	Arabic navigation and right-to-left presentation are implemented; participant usability results are not available.
NFR02	Access protection	Session-expiry and selected security-header behaviour are integration-tested.
NFR03	Data integrity	Database integrity and foreign-key checks run on the supplied demonstration database.
NFR04	Portability	The package includes a local launcher and a WSGI adapter.
NFR05	Recoverability	Backup functions and instructions exist; recovery-time objectives have not been benchmarked.
NFR06	Performance	The intended workload is a small pilot; no production throughput guarantee is asserted.
NFR07	Maintainability	Schema, test suite, configuration variables and deployment guide are supplied.

4.2.3 User requirements

The administrator needs a complete view of registration and finances and the ability to perform archive, receipt, debt and user-management actions. A teacher needs access to assigned student groups, marks and attendance without unrestricted financial administration. Parents and students need accurate records from staff, but their own account access is outside the current scope.

4.3 Data Collection and Requirements Elicitation

The evidence used consists of the supplied HTML interface, Python request handlers, SQLite schema, demonstration-data generator, test suite and project delivery notes. Requirements were extracted by following a user action from its interface control to the corresponding endpoint and database operation. Conflicting descriptions were reconciled against the implementation, including the financial currency labels.

No interview transcripts, questionnaire datasets or participant consent records are included. Therefore, stakeholder interviews and surveys are not claimed as completed methods. The administrative problem is modeled from the artifact and its documented workflows. A later school-based requirements study could validate whether the resulting priorities match staff needs.

The demonstration data are synthetic. They support repeatable examples and privacy-preserving testing, rather than estimates of real enrolment or fee collection. They must not be interpreted as a dataset sampled from actual children or families.

4.4 System Requirements and Feasibility

4.4.1 Software and hardware requirements

Local operation requires a compatible Python 3 installation, a modern browser and write access to the project database directory. The supplied guide specifies Python 3.10 or later. Excel support additionally requires openpyxl. Hosted operation requires a PythonAnywhere account, a configured WSGI application and persistent absolute paths for the database and backup directory.

For planning a small demonstration, a conventional computer with 4 GB of RAM and several hundred megabytes of free space is a reasonable starting assumption. This is

not a measured minimum specification. Storage growth depends on records, attachments and retained backups. The user must check available disk space and browser compatibility on the actual demonstration device.

4.4.2 Technical operational and economic feasibility

Technical feasibility is supported by successful local integration tests and the supplied hosted adapter. Operational feasibility depends on staff training, password changes, backups and clear responsibility for the authoritative database. Economic feasibility is qualitative: the core stack avoids mandatory database-server licensing, but hosting, Internet access, devices, support and staff time can still incur costs. No current price quotation or verified total cost of ownership is claimed.

4.5 System Modeling and Design

4.5.1 Use case diagram

The actors are administrator and teacher. The administrator manages records, financial actions, campaigns, users and backups. The teacher interacts with assigned classes and learning records. Authentication and assignment checks constrain the use cases; a visible menu alone is not a permission grant.

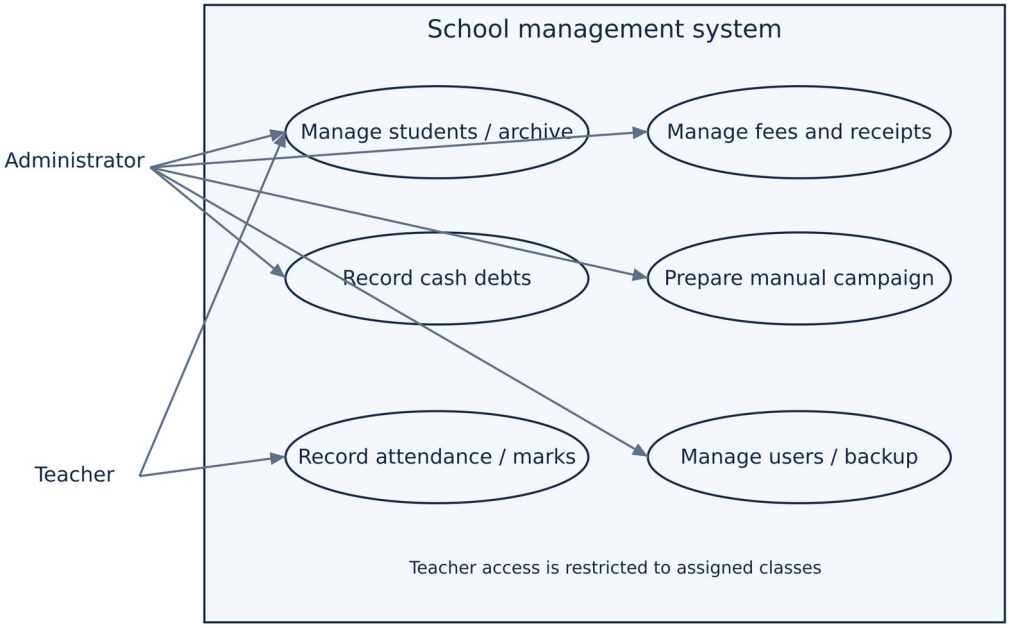


Figure 4.1 Use case diagram

4.5.2 Class diagram

The class diagram is a conceptual domain model of Student, MonthlyFee, Receipt, ClassMembership, CashDebt and DebtPayment. It describes responsibilities and associations used in analysis. The Python implementation is largely handler-based, so these domain boxes must not be mistaken for six implemented Python class declarations. APIHandler is an actual implementation class.

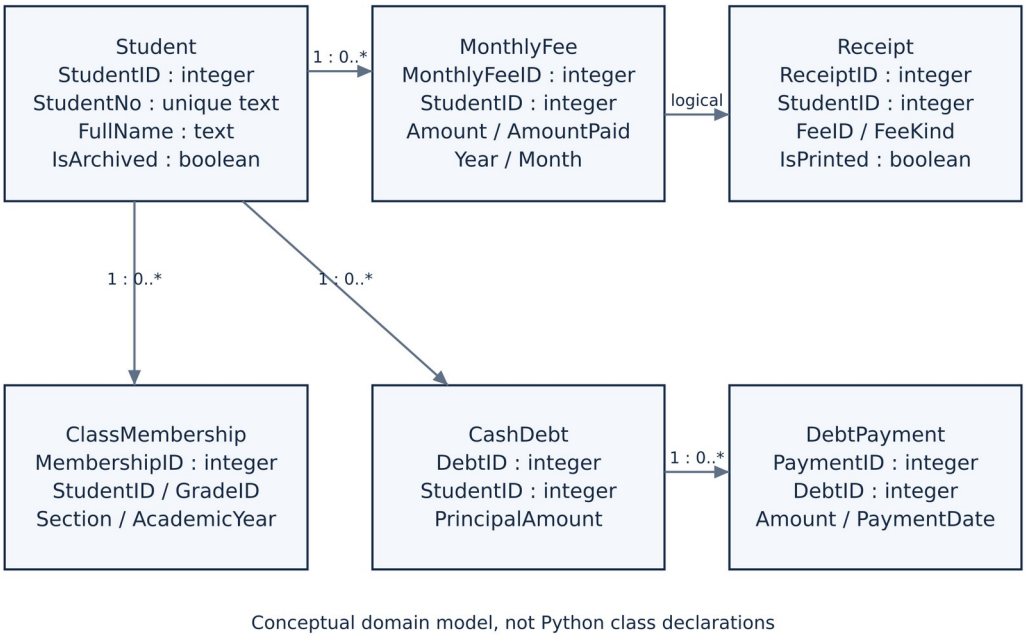


Figure 4.2 Conceptual domain class diagram

4.5.3 Sequence diagram

The receipt sequence follows payment entry, authenticated API handling, draft insertion, response and a separate print-confirmation request. A database write records the confirmation state. The diagram does not claim that browser print preview can verify a physical printer or paper receipt.

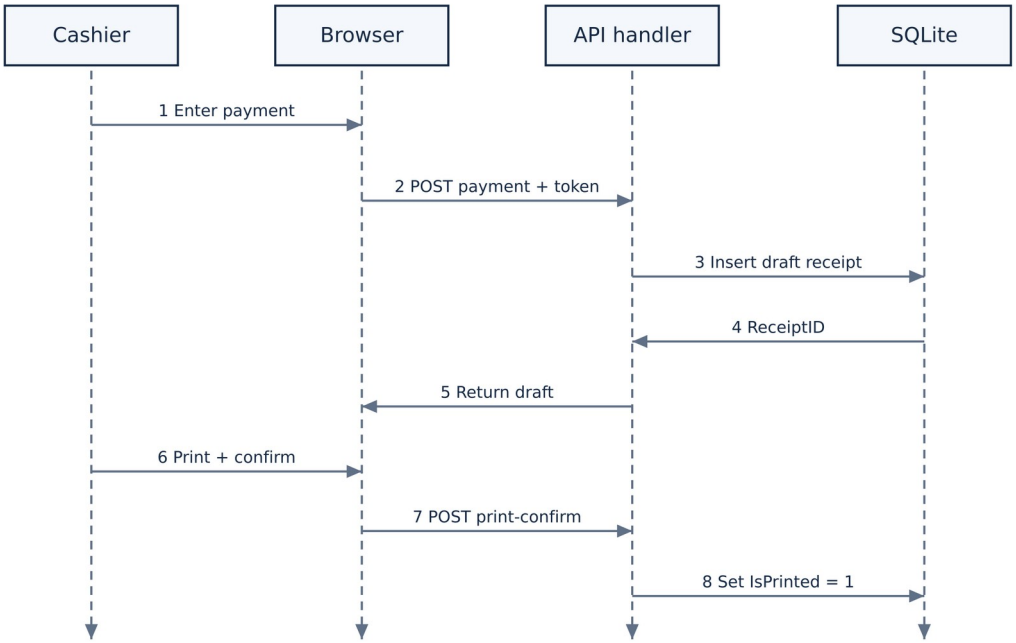


Figure 4.3 Payment and receipt confirmation sequence

4.5.4 State diagram

Before a payment there is no receipt. Recording the payment creates a receipt with IsPrinted set to zero. The explicit confirmation action changes that field to one. The model intentionally does not introduce a cancellation or reverse-state transition that was not verified in this workflow.

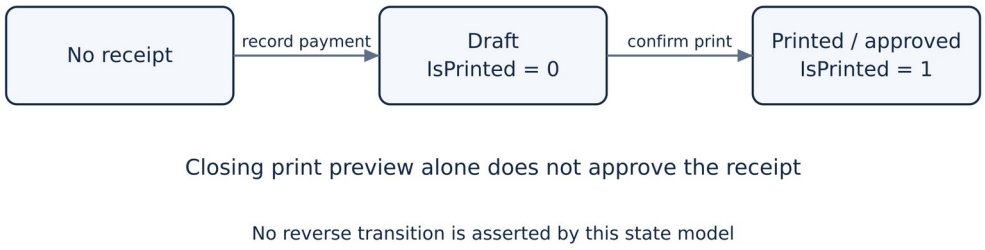


Figure 4.4 Receipt state diagram

4.5.5 Activity diagram

The communication activity starts with filters, continues through list review and requires confirmation before storage. Declining confirmation returns the user to review. After the campaign is stored, actual messaging remains outside the application. No branch in the diagram claims automatic transmission or delivery acknowledgement.

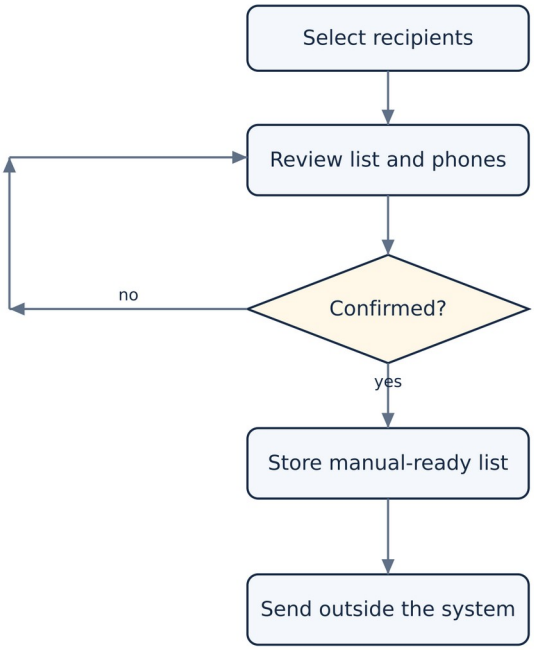


Figure 4.5 Manual communication activity diagram

4.6 Database Design

The schema contains 36 application tables, excluding SQLite's internal sequence table. StudentID is the main identity used across student-related modules. The selective ERD shows important physical relationships, while Appendix B exports the full executable schema and a table inventory.

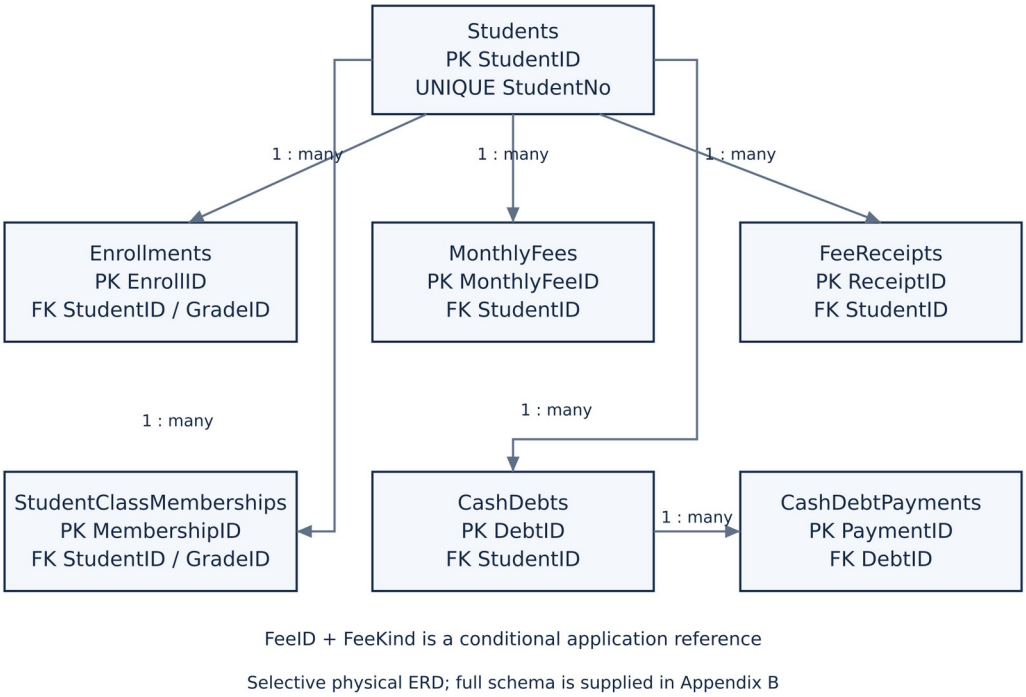


Figure 4.6 Selective physical entity relationship diagram

Entity	Key and relationship	Design responsibility
Students	PK StudentID; UNIQUE StudentNo	Identity, contact and archive fields
Enrollments	PK EnrollID; FK StudentID, GradeID, SchoolID	Primary academic placement
StudentClassMemberships	PK MembershipID; FK StudentID and GradeID	Additional group membership, unique per student/group/year
MonthlyFees	PK MonthlyFeeID; FK StudentID	Unique student, year and month fee row
FeeReceipts	PK ReceiptID; FK StudentID	Payment evidence and printed state
CashDebts	PK DebtID; FK StudentID	Positive debt principal
CashDebtPayments	PK PaymentID; FK DebtID	Positive payment entries for a debt
ParentStudents	Composite primary key; UNIQUE StudentID	The current schema permits at most one parent link per student

Entity	Key and relationship	Design responsibility
UserSessions	PK Token; FK UserID	Authenticated session with expiry metadata

Two limitations are important. First, ParentStudents has a unique constraint on StudentID, so the delivered schema is not an unrestricted many-parent relationship. Second, FeeReceipts uses FeeKind together with FeeID to refer to monthly or additional fees. That conditional relationship is enforced in application logic rather than a single foreign key to two possible tables. The ERD therefore does not draw an unconditional physical foreign key from FeeID to MonthlyFees.

Some monetary columns use SQLite REAL. This is adequate for the current demonstration but can introduce binary floating-point rounding concerns in institutional accounting. A future revision should use integer minor units or a carefully specified decimal representation and migrate existing values with reconciliation tests.

4.7 User Interface Design

The interface uses a right-side navigation area, a module heading, filters and record tables. Student identity remains visible when moving from a student to a financial action. Potentially consequential operations require specific actions rather than being inferred from closing a dialog. The following wireframe shows the intended information structure, and is explicitly a design model rather than a screenshot.

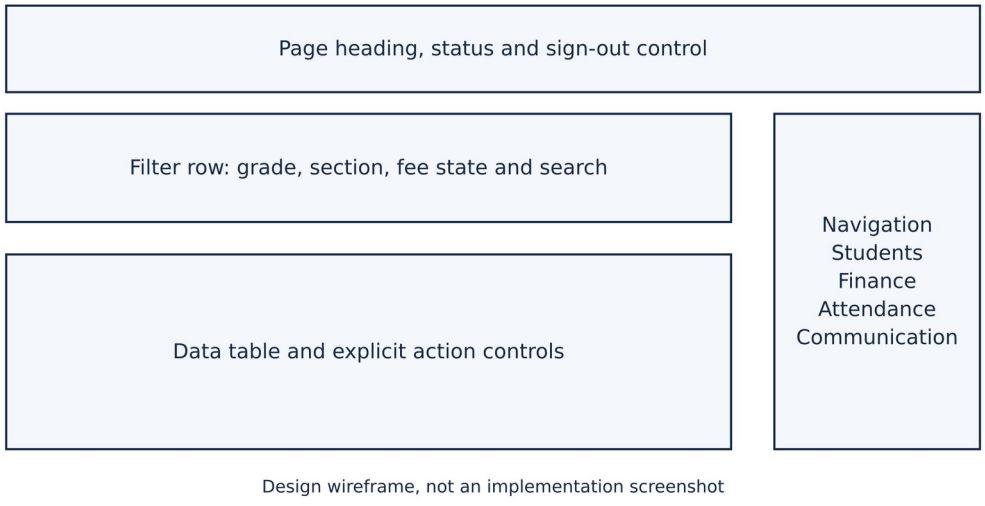


Figure 4.7 Interface layout wireframe

4.8 System Implementation

The main implementation resides in index.html, server.py and database/init_db.py. The browser renders modules and sends JSON requests. APIHandler routes HTTP methods and paths, checks sessions and applies validation. The database module creates tables, indexes and migrations. Tests construct a temporary database and communicate with the local HTTP handler, while wsgi.py adapts hosted requests.

Archive implementation records archive state, time, reason and note. Active lists filter archived students rather than physically deleting their records. Additional memberships use StudentClassMemberships and preserve the student's primary identity. Cash debt payments append separate entries rather than overwriting the original principal.

Passwords use salted PBKDF2-HMAC-SHA256 with 600000 iterations in the delivered implementation. Protected requests validate session tokens and their expiry. The application emits selected defensive headers, and hosted mode includes HSTS. These controls are concrete implementation features, not a certification that every endpoint is free from vulnerabilities. Independent review remains appropriate before using real institutional data.

The local listener defaults to 127.0.0.1. The hosted entry point is the WSGI application behind PythonAnywhere's web server. Configuration variables include AFAQ_DB_PATH, AFAQ_BACKUP_DIR, AFAQ_CLOUD and AFAQ_SQLITE_WAL. The AFAQ prefix is an internal configuration identifier retained for compatibility and does not change the displayed project title.

4.9 Database Implementation

database/init_db.py creates or upgrades the schema. Parameterized operations bind data separately from SQL syntax. MonthlyFees uses a unique combination of StudentID, Year and Month. StudentClassMemberships prevents a repeated student, grade, section and academic-year combination. Positive-value checks constrain debt principal and payment rows. Application validation supplies rules that cannot be expressed by these constraints alone.

The supplied students.db is a demonstration database, not a production database export. tools/create_demo_data.py can regenerate its fictional records, and the reset option intentionally replaces the demonstration database. Backups must be made

before regenerating any database whose contents need to be retained. Appendix B includes schema.sql and an inventory of columns and foreign keys for reproducibility.

4.10 User Interface Implementation

The implemented Arabic login screen is shown in Figure 4.8. It identifies the project and provides username and password fields. The capture was taken from the hosted pilot on 28 October 2026. It documents the actual entry screen, not a redesigned mockup.

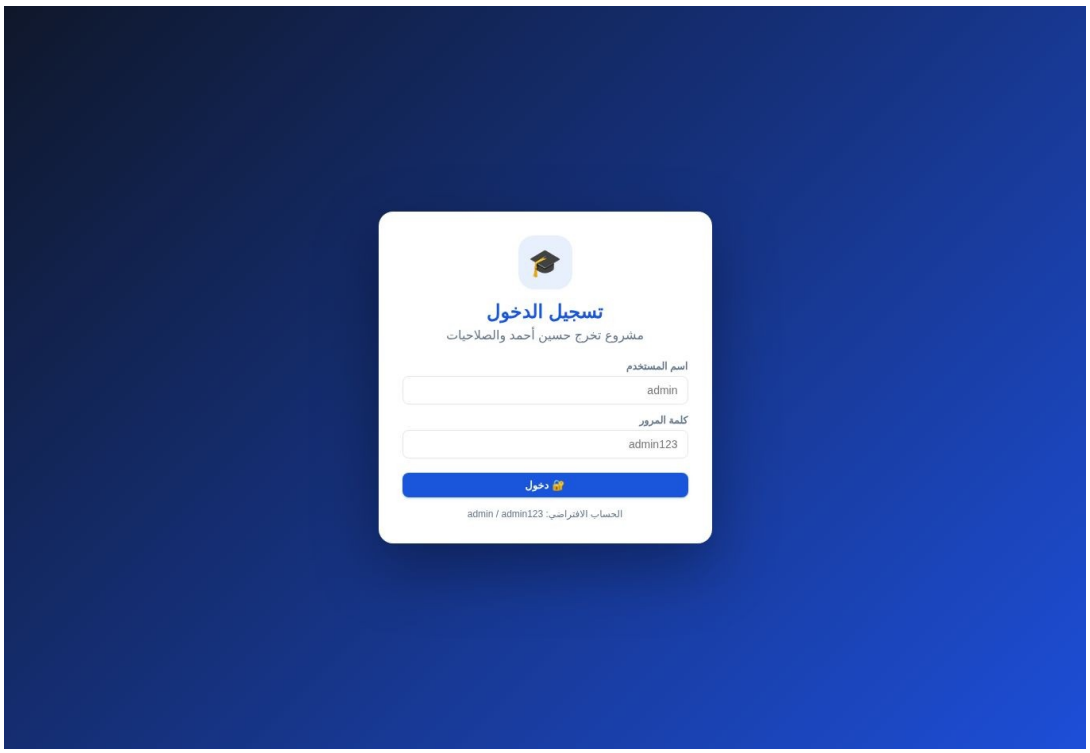


Figure 4.8 Actual hosted Arabic login screen

After successful authentication, index.html provides student, fee, attendance, marks, membership and manual-message modules. The administrator selects a module, applies filters, reviews records and uses the relevant explicit action. For receipts, the workflow records a payment before approving the receipt through the print-confirm action. For communication, the interface prepares a reviewed list for manual sending. The current visual evidence contains the entry screen. Authenticated-screen coverage is a remaining documentation task and should be completed in the student's own demonstration session before final submission. It must use the fictional demonstration database and must not expose real student information. Appendix C provides a capture checklist for the student list, fees, receipt and manual-campaign screens.

4.11 System Testing

4.11.1 Test environment and procedure

The integration command is python -m unittest discover -s tests -v, executed from the project directory. The suite creates a temporary SQLite database and starts a local HTTP server on an automatically selected port. It signs in through the API and sends requests against the same handler used by the application. Each scenario receives reset test data, so assertions do not depend on the distributed demonstration database.

The suite completed with four passing tests on 28 October 2026. A second database check returned integrity_check = ok and no foreign_key_check violations for the supplied demonstration database. The WSGI scenario checks the adapter through a constructed WSGI environment. It is not a remote load test or proof that the currently hosted account has every setting configured correctly.

Test case	Observed assertions	Result
Archive integrity	Empty reason rejected; archived record remains retrievable; active finance excludes it	Pass
Health and session expiry	Health returns success; framing denied; expired token rejected	Pass
Combined business workflow	Extra membership, debt payment, confirmed manual campaign and receipt print gate	Pass
PythonAnywhere WSGI adapter	Health response, hosted HSTS header and authenticated user route	Pass

4.11.2 Requirements traceability

Requirement	Evidence path	Coverage boundary
FR02 archive	test_archive_requires_reason_and_hides_student_from_active_finance	Tests reason and active-list exclusion
FR03 membership	test_membership_debt_manual_campaign_and_printed_receipt	Tests one added membership and retrieval

Requirement	Evidence path	Coverage boundary
FR05 receipt	Same combined workflow scenario	Tests draft exclusion and print-confirm inclusion
FR06 debt	Same combined workflow scenario	Tests one debt and a partial payment
FR07 communication	Same combined workflow scenario	Tests explicit confirmation and manual-ready record
FR09 sessions	test_health_headers_and_session_expiry	Tests expiry rejection and selected headers
O6 deployment	test_pythonanywhere_wsgi_adapter	Tests constructed WSGI requests, not live capacity

4.11.3 Interpretation and remaining evaluation

Four passing scenarios show that the assertions exercised by those scenarios behaved as expected. They do not imply 100 percent code coverage, complete permission-matrix coverage or production readiness. There are no measured usability, accessibility, penetration, stress or disaster-recovery results. Dedicated unit tests for every calculation and exhaustive endpoint tests remain future work.

A useful next evaluation would recruit representative staff, define registration and receipt tasks, measure successful completion, errors and time, and record consent before collecting responses. A capacity test should separately measure concurrent writes and response-time distributions. These are recommended protocols, not fabricated results from completed experiments.

4.12 Chapter Summary

The design ties student identity to enrolment, learning and financial records. The required models explain actors, domain relationships, receipt interactions and communication control. The implementation passes the four supplied integration scenarios and database integrity checks. Important remaining limits include authenticated-screen documentation, participant evaluation, concurrency measurements and independent security assessment.

CHAPTER FIVE CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter summarizes what the implemented artifact and its evaluation establish. It distinguishes the contributions demonstrated in the package from the work still needed for a final institutional deployment and from research questions requiring participant evidence.

5.2 Summary of Key Findings

The application integrates student records, additional memberships, attendance, marks and financial workflows under a consistent student identity. The archive rule preserves records while changing visibility in active views. The receipt rule distinguishes a draft record from a print-confirmed record, and the debt ledger preserves a principal with separate payment entries. The communication module records a confirmed recipient list for manual use.

The local HTTP mode and WSGI adapter support different deployment environments without a second implementation of the main rules. Four integration scenarios passed, and the distributed demonstration database passed integrity and foreign-key checks. The evidence supports selected functional correctness and technical feasibility. It does not establish improved staff productivity, universal usability, institutional security or high-volume performance.

5.3 Contributions to the Field

The project contributes an inspectable Arabic-first software artifact, a requirement-to-test mapping and a documented combination of local and hosted operation. The explicit receipt and manual-communication states are useful examples of converting administrative meanings into enforceable software behaviour. The supplied fictional data and isolated tests make the demonstration reproducible without distributing a real children's database.

These contributions are practical and educational. They do not constitute a claim that the prototype replaces established school platforms or introduces globally novel school-management functions. Its relevance is the transparent implementation and evaluation of the bounded workflow requirements described in this report.

5.4 Recommendations

Before final submission, capture the authenticated demonstration screens, review the report with the supervisor, import the supplied reference records into Mendeley and insert or verify citations through Mendeley Cite. The student and supervisor should complete only their own signature and approval fields. The code, report, presentation and supporting evidence should be placed in the designated GitHub repository without production credentials or real student data.

Before operational adoption, replace demonstration passwords, designate the authoritative database, test backup restoration and train users on archive and receipt semantics. Record who is responsible for maintenance and recovery. Review every sensitive endpoint against the role matrix and migrate monetary values to a precise representation before relying on the application for institutional accounting.

5.5 Recommendations for Future Research

Future research should examine task completion and error rates with representative Arabic-speaking school staff, using a defined protocol and appropriate consent. A comparative study could evaluate the prototype against a configured existing system using the same tasks rather than comparing marketing descriptions. Such a study should report sample characteristics and missing data alongside its results.

Further engineering research can investigate concurrent-write behaviour, migration to PostgreSQL, multiple-guardian modeling and safe synchronization. Synchronization should be treated as a separate conflict-resolution problem with an explicit authoritative-record policy. Communication integration, if later introduced, would need consent, audit records and truthful provider delivery states. None of these future functions is represented as completed in this project.

REFERENCES

Gibbon (n.d.). *Gibbon the flexible school platform.* Retrieved October 28, 2026, from https://gibbonedu.org/

openSIS (n.d.). *Student management software features.* Retrieved October 28, 2026, from https://opensis.com/features

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24(3), 45–77.* https://doi.org/10.2753/MIS0742-1222240302

Python Software Foundation (n.d.). *sqlite3 — DB-API 2.0 interface for SQLite databases.* Retrieved October 28, 2026, from https://docs.python.org/3/library/sqlite3.html

PythonAnywhere (n.d.). *Webapp basics.* Retrieved October 28, 2026, from https://help.pythonanywhere.com/pages/WebAppBasics/

SQLite (n.d.). *Appropriate uses for SQLite.* Retrieved October 28, 2026, from https://www.sqlite.org/whentouse.html

Telem, M., & Avidov, O. (1995). The effect of school management information systems on the nature of a loosely coupled high school instruction–administration subsystem: A preliminary study. *Journal of Research on Computing in Education, 28(2), 258–269.* https://doi.org/10.1080/08886504.1995.10782164

Visscher, A. J., Wild, P., Smith, D. D., & Newton, L. (2003). Evaluation of the implementation, use and effects of a computerised management information system in English secondary schools. *British Journal of Educational Technology, 34(3), 357–366.* https://doi.org/10.1111/1467-8535.00332

APPENDICES

Appendix A Reproducible execution

From the code directory, run python server.py for local operation and python -m unittest discover -s tests -v for the isolated integration suite. The Windows launcher is START_OFFLINE.bat. The four passing cases and their assertions are described in Section 4.11. The evidence folder contains the test transcript and database verification results.

PythonAnywhere uses the supplied wsgi.py and pythonanywhere_wsgi.example.py. Configure absolute database and backup paths, retain the project directory on sys.path and reload the web app after deployment. Preserve the original database before any migration or demonstration-data regeneration.

Appendix B Database inventory and supporting code

Table	Columns	Declared foreign keys
ActivityLogs	9	0
Aids	7	1
AppSettings	2	0
Attendance	8	2
CashDebtPayments	7	1
CashDebts	9	2
CommunicationCampaigns	8	1
CommunicationRecipients	5	2
Enrollments	6	3
FeeReceipts	12	1
Fees	10	1
FinancialCategories	2	0

Table	Columns	Declared foreign keys
FinancialStatus	5	2
Gender	2	0
Governorates	2	0
GradeSubjects	3	2
Grades	2	0
GraduationProjects	12	2
HealthStatus	5	1
MonthlyFees	10	1
Orphans	6	1
ParentStudents	2	2
Parents	6	0
Schools	5	1
Status	2	0
StudentClassMemberships	7	2
StudentFinanceAccounts	3	1
StudentFinanceLedger	11	1
StudentMarks	10	2
Students	19	3
Subjects	3	0
TeacherGrades	4	2
TeacherProfiles	7	1
TeacherSubjects	3	2
UserSessions	4	1

Table	Columns	Declared foreign keys
Users	9	0

The complete executable table and index definitions are in schema.sql. Source files include server.py, index.html, wsgi.py, database/init_db.py, tools/create_demo_data.py and tests/test_system.py. The domain class diagram is an analysis abstraction. Conditional fee references and the unique parent-student constraint should be reviewed before extending the schema.

Appendix C Submission and evidence checklist

The submission package contains the Word report, matching PDF, Arabic presentation, source code, fictional demonstration database, test evidence, schema and editable SVG diagrams. The source SVG diagrams can be imported into draw.io for further editing; they are not generated by an alleged field study.

Before the final discussion, capture the authenticated dashboard, student list, monthly-fee table, draft and approved receipt, and confirmed manual campaign using fictional records. Check that account tokens and real personal information are not visible. Confirm receipt state with the API or database rather than inferring it from the appearance of the print preview.

Import References_Mendeley.ris into Mendeley Reference Manager. In Word, use Mendeley Cite to insert or verify each author-date citation and choose APA 7th edition for the bibliography. The supplied document uses readable APA citations; it does not claim that Mendeley Cite fields have already been inserted through an authenticated Mendeley session.

Create or select a GitHub repository dedicated to this graduation project. Upload the source code and docs folder, retain tests and evidence, and exclude Python caches, backups, production databases and secrets. The accompanying local repository and upload guide provide the prepared structure. Record the actual repository URL only after the upload succeeds.